School of Future Tech

Case Study Report

on

Airline Reservation System

by

Samarth Sunil Navale

150096725148

Index

1. Introduction to the Case Study.
2. Problem Statement / Case Background (Abstract).
3. Problem Statement / Case Study Design.
4. Methods & Algorithms Technology Applied in the Problem Statement / Case Study.
5. Problem Statement / Case Study Implementation Details and Snapshots.
6. Problem Statement / Case Study Results and Conclusion.
7. References

1. Introduction to the Case Study

Airline reservation systems are essential for managing passenger bookings, flight schedules, and seat availability in an organized manner. These systems allow airlines to allocate seats efficiently, prevent overbooking, and maintain records of passenger reservations.

This case study focuses on designing and implementing a **basic Airline Reservation System using C++**. The system allows users to view available flights, check seat availability, book seats for passengers, cancel reservations, and view booking details through a menu-driven interface.

The case study demonstrates how programming concepts such as **classes, arrays, conditional logic, and input validation** can be used to simulate real-world reservation workflows. It highlights the importance of proper seat allocation, booking management, and system organization in building a reliable reservation system.

2. Problem Statement / Case Background (Abstract)

Background

Managing airline reservations manually can lead to problems such as incorrect seat allocation, double bookings, and difficulty tracking passenger reservations. As the number of passengers and flights increases, maintaining accurate booking records and seat availability becomes more challenging. A structured reservation system helps organize flight data, allocate seats efficiently, and manage booking operations in a reliable manner.

Abstract

This case study presents the design and implementation of a **console-based Airline Reservation System using C++**. The system manages multiple flights, each with a fixed seat capacity, and allows users to perform essential reservation operations such as viewing flight schedules, booking seats, cancelling reservations using a PNR, and checking seat availability. The program uses object-oriented programming concepts, arrays, and conditional logic to simulate a simplified airline booking workflow. The system ensures proper seat allocation, prevents overbooking, and dynamically updates seat status after every booking or cancellation. The implementation demonstrates how fundamental C++ programming concepts can be applied to build a basic reservation management system.

3. Case Study Design

The Flight Reservation System is designed as a modular pipeline with the following stages:

1. The **Airline Reservation System** is designed as a menu-driven console application that manages flight information, seat allocation, and passenger reservations. The system is structured into the following components:
2. **Flight Data Management**
   The system stores information for multiple flights, including **flight number, source city, destination city, and seat capacity**. Each flight maintains an array of seats to track whether a seat is booked and the passenger associated with that seat.
3. **Seat Allocation and Availability**
   Each flight contains a fixed number of seats. The system checks seat availability before allowing a booking and automatically assigns the next available seat. The seat status is updated dynamically after every booking or cancellation.
4. **Reservation Management**
   When a passenger books a seat, the system generates a unique **PNR (Passenger Name Record)** and stores booking details such as passenger name, flight number, and seat number. This information is used to track reservations and manage cancellations.
5. **Booking and Cancellation Operations**
   Users can perform reservation operations through a menu-driven interface. Bookings are created only when seats are available, and cancellations are processed using the PNR, which frees the corresponding seat.
6. **Passenger Booking Records**
   The system maintains booking records that allow users to view all reservations associated with a passenger's name. This helps simulate a basic passenger profile within the reservation system.
7. **System Interaction Flow**
   Users interact with the system through a console menu that allows them to **view flights, search flights by city, book seats, cancel bookings, view seat maps, and check their booking records**. The system continuously updates flight and seat information to ensure accurate reservation management.

4. Methods & Algorithms Technology Applied in the Problem Statement / Case Study

Key methods and algorithms:

Object-Oriented Programming (OOP)
The system is designed using classes such as **Flight** and **Seat** to represent real-world entities. This approach helps organize flight data, seat information, and booking status in a structured manner.

Array-Based Data Management
Arrays are used to store flights, seats, and booking records. Fixed-size arrays manage flight lists, seat availability, and passenger reservations during program execution.

Seat Allocation Logic
The system checks seat availability before booking. When a booking is made, the program assigns the **first available seat** and updates its status as booked. If the flight is full, booking is not allowed.

PNR Generation Algorithm
Each reservation is assigned a unique **PNR (Passenger Name Record)** generated using an incremental counter. This PNR is used to track and cancel bookings.

Booking Validation and Constraints
The system includes validation checks such as:

- Preventing booking on non-existent flights
- Ensuring seats are available before allocation
- Limiting the number of bookings per passenger on a flight

Menu-Driven Interaction
The program uses a loop-based menu system that allows users to repeatedly perform operations such as viewing flights, booking seats, cancelling reservations, and viewing seat maps.

Technology Stack Used for the Case

Programming Language
C++

Tools / Concepts Used

- Standard C++ libraries (`iostream`, `string`)
- Classes and objects
- Arrays for data storage
- Conditional statements and loops
- Input validation for user interaction

Development Environment
The program can be developed and executed using any C++ development environment, such as **VS Code, Code::Blocks, Dev-C++, or a standard C++ compiler**.

Data Storage
All data, such as flight details, seat status, and booking records are stored **temporarily in memory using arrays during program execution**. The data resets when the program terminates.

5. Problem Statement / Case Study Implementation Details and Snapshots.

The **Airline Reservation System** is implemented as a **console-based C++ program** that simulates basic airline booking operations. The program is organized using classes, arrays, and functions to manage flights, seats, and passenger reservations.

Program Structure

Seat Class
Stores seat information, including:

- Booking status (booked or available)
- Passenger name associated with the seat

Flight Class
Stores flight-related data, including:

- Flight number
- Source and destination cities
- Seat capacity
- An array of seat objects

The class also includes functions to:

- Display flight information
- Count available seats
- Show the seat map of the flight

Booking Management
Passenger bookings are stored using arrays that track:

- PNR number
- Flight number
- Seat number
- Passenger name

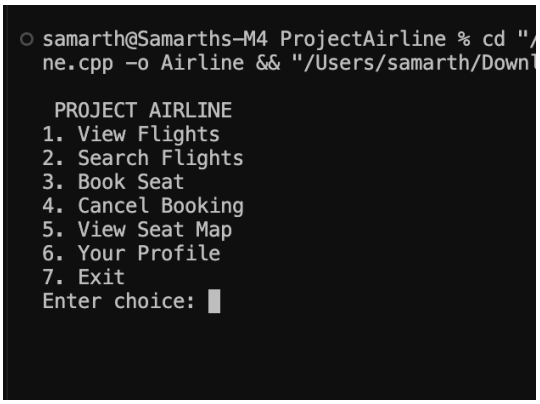
Main System Operations

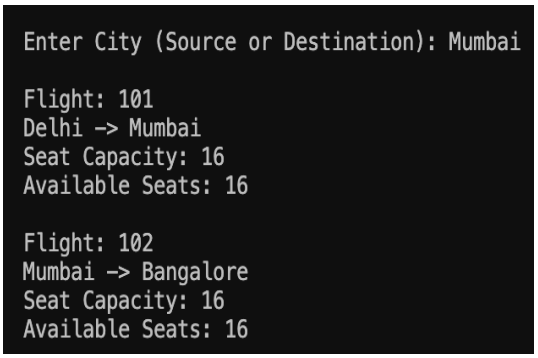
The program provides a **menu-driven interface** that allows the user to perform the following operations:

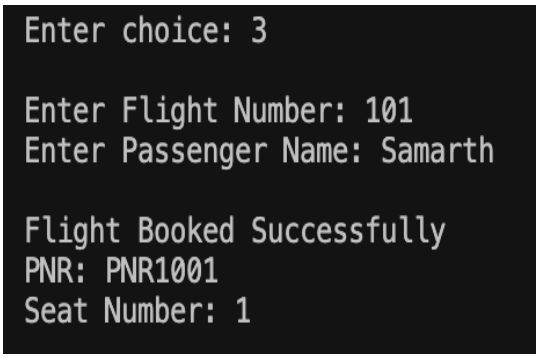
- View all available flights
- Search flights by source or destination
- Book a seat on a selected flight
- Cancel an existing booking using a PNR
- View the seat map of a flight
- View passenger booking details

The system continuously updates seat status and booking records whenever a booking or cancellation occurs.

Snapshots:







```
Enter choice: 5

Enter Flight Number: 101

Seat Map for Flight 101:

 1[X]   2[ ]   3[ ]   4[ ]
 5[ ]   6[ ]   7[ ]   8[ ]

 9[ ]  10[ ]  11[ ]  12[ ]
13[ ]  14[ ]  15[ ]  16[ ]



X = Booked | Empty = Available
```

```
 PROJECT AIRLINE
1. View Flights
2. Search Flights
3. Book Seat
4. Cancel Booking
5. View Seat Map
6. Your Profile
7. Exit
Enter choice: 6

Enter Passenger Name: Samarth

--- Bookings for Samarth ---
PNR: PNR1001 | Flight: 101 | Seat: 1
```

6. Problem Statement / Case Study Results and Conclusion.

Findings

The system successfully manages flight information, seat allocation, and passenger reservations. Seats are booked only when available, preventing overbooking. The use of unique **PNR numbers** allows bookings to be tracked and cancelled easily, while seat availability updates automatically after each operation.

Conclusion

This case study demonstrates a **menu-driven Airline Reservation System using C++** that integrates flight management, seat booking, and cancellation operations. It shows how basic programming concepts such as **classes, arrays, loops, and conditional logic** can be used to build a simple reservation management system.

7. References

- Official C++ documentation and standard library resources.
- Online C++ programming tutorials and learning materials.
- Documentation and guides related to object-oriented programming in C++.
- Educational articles on implementing console-based systems using C++.
- Programming textbooks covering C++ fundamentals, arrays, and classes.
- Online technical resources explaining menu-driven program design and basic data management techniques.